DeepAgents 框架学习大纲

基于《DeepAgents_框架介绍与应用实践》整理的学习路径

目 录目 录

第一阶段：框架定位与核心价值

第二阶段：与 LangChain 及 LangGraph 的区别

第三阶段：核心功能介绍

第四阶段：四大核心内置工具与组件

第五阶段：文件系统集成

第六阶段：总结与最佳实践

第一阶段：框架定位与核心价值

1.1 什么是 DeepAgents？

定义：基于 LangChain 和 LangGraph 构建的企业级高级智能体框架

定位：高阶的 Agent Harness（智能体装备/套件），建立在 LangGraph（底层运行时）和 LangChain（工具/模型层）之上

核心理念：如果说 LangChain 提供了积木，LangGraph 提供了地基，那么 DeepAgents 就是一套成品级的框架。它预设了最佳实践（规划、文件系统、子智能体），让你能快速构建类似 "OpenAI Deep Research" 或 "Claude Code" 的应用。

开源地址：https://github.com/langchain-ai/deepagents

论文地址：https://arxiv.org/pdf/2510.21618

【注：框架架构图详见原始文档】

1.2 解决的核心问题

传统 Agent 开发运行简单的循环：思考→调用工具→观察→重复，容易遇到以下"浅层陷阱"：

问题	描述
规划能力缺失	原生 Agent 倾向于"走一步看一步"，缺乏全局视角的任务拆解
遗忘与混乱	长任务中容易忘记初始目标或陷入死循环

环境交互困难	文件系统操作、代码执行环境的配置和安全管理复杂
上下文污染	所有工具返回结果堆积在对话历史中，噪声过大
协作编排复杂	多智能体之间的任务分发和上下文隔离难以实现

1.3 应用场景

场景类型	能力说明	代表性案例
深度调研	自动进行多轮网页搜索、阅读文档、整理笔记并生成长篇报告	OpenAI Deep Research
全栈代码生成	在沙箱环境中编写、运行、测试和修复代码	Claude Code、Manus
复杂数据分析	自动连接数据库，生成 SQL，执行查询，生成图表	数据分析智能体
自动化运维	操作文件系统、执行 Shell 命令、管理服务器状态	DevOps 自动化
复杂工作流编排	多角色协作的复杂业务流程	企业流程自动化

第二阶段：与 LangChain 及 LangGraph 的区别

2.1 技术定位对比

特性	LangChain	LangGraph	DeepAgents
层级	基础组件库 (Foundation)	编排引擎 (Orchestration)	应用框架
核心抽象	Chain, Runnable, Tool	StateGraph, Node, Edge	DeepAgent, Middleware, Backend
灵活性	极高（积木块）	高（自定义图结构）	中（约定优于配置）

开箱即用	低（需自行组装）	中（需定义图逻辑）	高（内置规划、文件系统、子代理）
适用对象	库开发者/底层构建	复杂流程序列化开发者	应用开发者/企业级解决方案

2.2 层次关系

LangChain（积木）→ LangGraph（地基+连接）→ DeepAgents（成品框架）

LangChain：提供 Prompt，Models，Tools 等积木

LangGraph：提供 State，Nodes，Edges 等地基和连接逻辑

DeepAgents：LangGraph 的一种"最佳实践实现"，使用 LangGraph 管理状态和循环，隐藏底层图构建细节

第三阶段：核心功能介绍

3.1 安装环境与依赖

```
pip install deepagents
```

核心依赖：

- langchain ≥ 1.1.0
- langchain-core ≥ 1.1.0
- langgraph ≥ 1.0.2
- anthropic ≥ 0.73.0

3.2 核心入口：create_deep_agent()

```
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o", temperature=0)
agent = create_deep_agent(model=model)
```

关键参数

参数	说明
model	支持自定义语言模型
tools	自定义工具集
system_prompt	系统提示词

subagents	子代理配置
backend	文件存储后端
interrupt_on	人机交互配置，允许在特定节点暂停 Agent 执行

第四阶段：四大核心内置工具与组件

4.1 系统提示词 (System Prompts)

功能：定义 Agent 的"人设"、行为准则和核心目标

机制：框架自动将用户定义的 system_prompt 与内置的 BASE_AGENT_PROMPT 结合

默认提示词包含：

- 规划指令：要求 LLM 在任务开始前必须调用 write_todos
- 文件操作规范：明确 write_file 用于新内容，edit_file 用于局部修改
- 子代理调用协议：规定 task 工具的参数结构
- 安全底线：禁止直接执行删除等危险命令

4.2 规划工具 (Todo List)

组件：TodoListMiddleware

工具名：write_todos

工作流：

1. 接收复杂任务
2. 调用 write_todos 拆解为子步骤 (Pending)
3. 每完成一步，更新状态为 (Completed)
4. 自我反思：Agent 在每一步行动前看到 Todo List，避免迷失方向

4.3 文件系统 (File System)

组件：FilesystemMiddleware & Backend

功能：Agent 的"外挂大脑"，将长文本写入文件系统而非塞入上下文

核心工具集：

ls：浏览目录结构

read_file：读取文件内容（支持分页）

write_file：创建新文件

edit_file：修改文件（防呆设计）

glob：通配符模糊查找

grep：正则表达式搜索

execute：执行 Shell 命令

亮点功能：大结果自动转存，当工具返回结果过长时自动保存到文件系统

4.4 子代理 (Sub Agents)

组件：SubAgentMiddleware

工具名：task

功能：动态生成同构或异构子 Agent，实现上下文隔离与专业分工

使用场景：

- 高风险操作隔离
- 专业化分工
- 资源优化（子 Agent 可使用轻量化模型）

第五阶段：文件系统集成

5.1 后端支持 (Backends)

后端	说明
FilesystemBackend	直接操作本地磁盘
DockerBackend	在 Docker 容器中执行，提供隔离环境
StoreBackend	存储在数据库中，支持查询和检索
E2BBackend	使用 E2B 云端沙箱
CompositeBackend	混合模式（如：本地存文件，Docker 跑代码）

5.2 核心工具使用示例

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o", temperature=0)
backend = FilesystemBackend(root_dir="./workspace", virtual_mode=True)

agent = create_deep_agent(
    model=model,
    backend=backend,
    system_prompt="你是一个文件系统操作助手。"
)
```

第六阶段：总结与最佳实践

6.1 核心中间件总结

中间件	功能
FilesystemMiddleware	提供文件操作工具，大结果自动转存
TodoListMiddleware	拦截 LLM 输出的 todo_list，支持任务 CRUD
SubAgentMiddleware	自动创建 task 工具，支持通用分身和专家分身
HumanInTheLoopMiddleware	通过 interrupt_on 配置，支持人工审批
PatchToolCallsMiddleware	自动检测并修复"悬空"工具调用

6.2 一句话总结

DeepAgents 是 LangGraph 的最佳实践参考实现，它将构建"类人智能体"所需的规划、记忆、工具使用等复杂逻辑，封装成了标准化的基础设施，让开发者专注于业务逻辑而非底层编排。

6.3 学习路径建议

入门：理解框架定位，对比 LangChain/LangGraph

基础：掌握 create_deep_agent() 核心 API

进阶：深入学习四大中间件的原理和使用

实战：结合具体场景进行项目实践

参考资源

GitHub：https://github.com/langchain-ai/deepagents

论文：https://arxiv.org/pdf/2510.21618

官方文档：https://python.langchain.com/docs/tutorials/

本大纲基于《DeepAgents_框架介绍与应用实践》整理